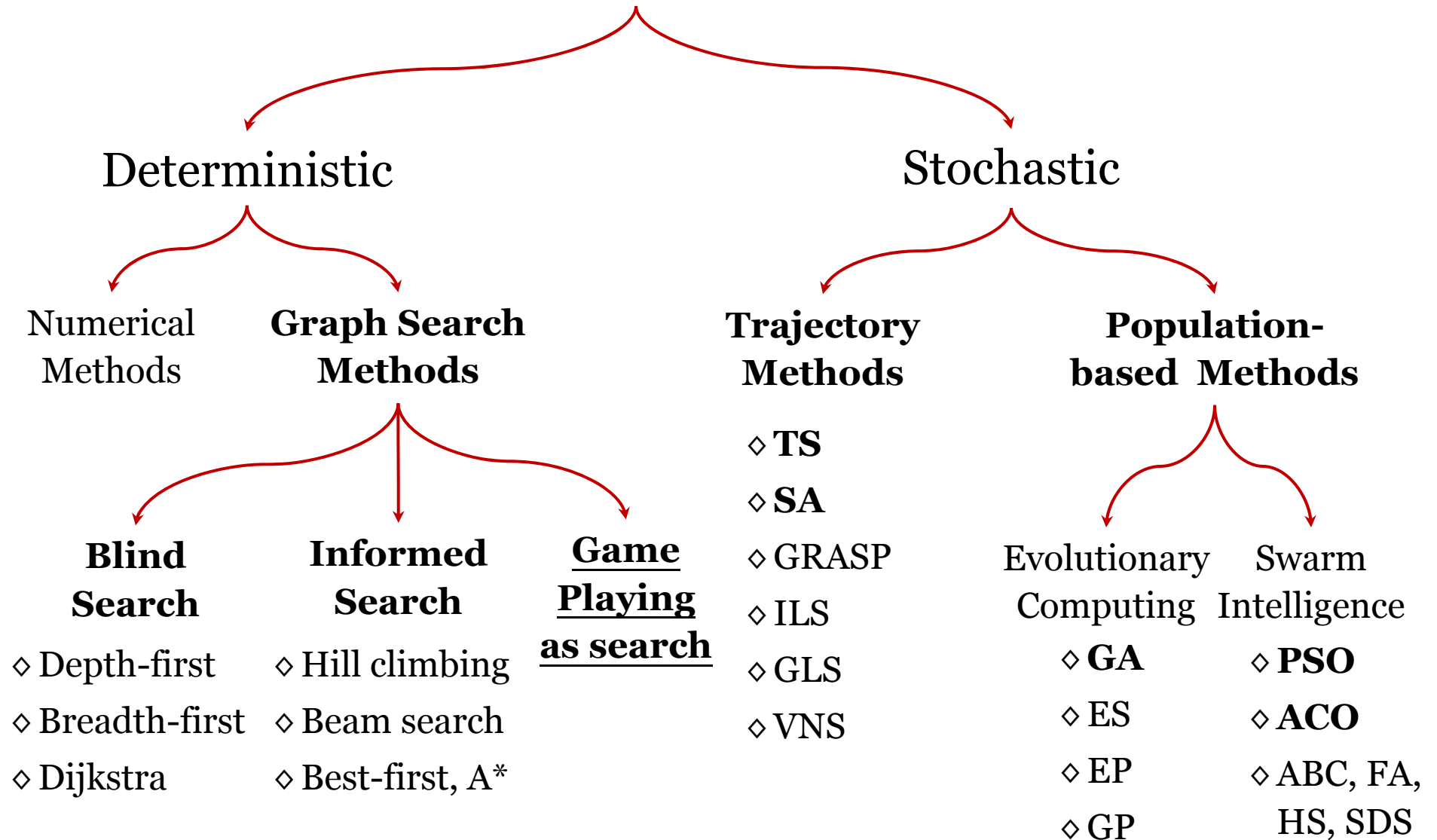


Game Playing as Search

Lecture 3 – Thursday May 15, 2014

Outline

Search Methods



Outline

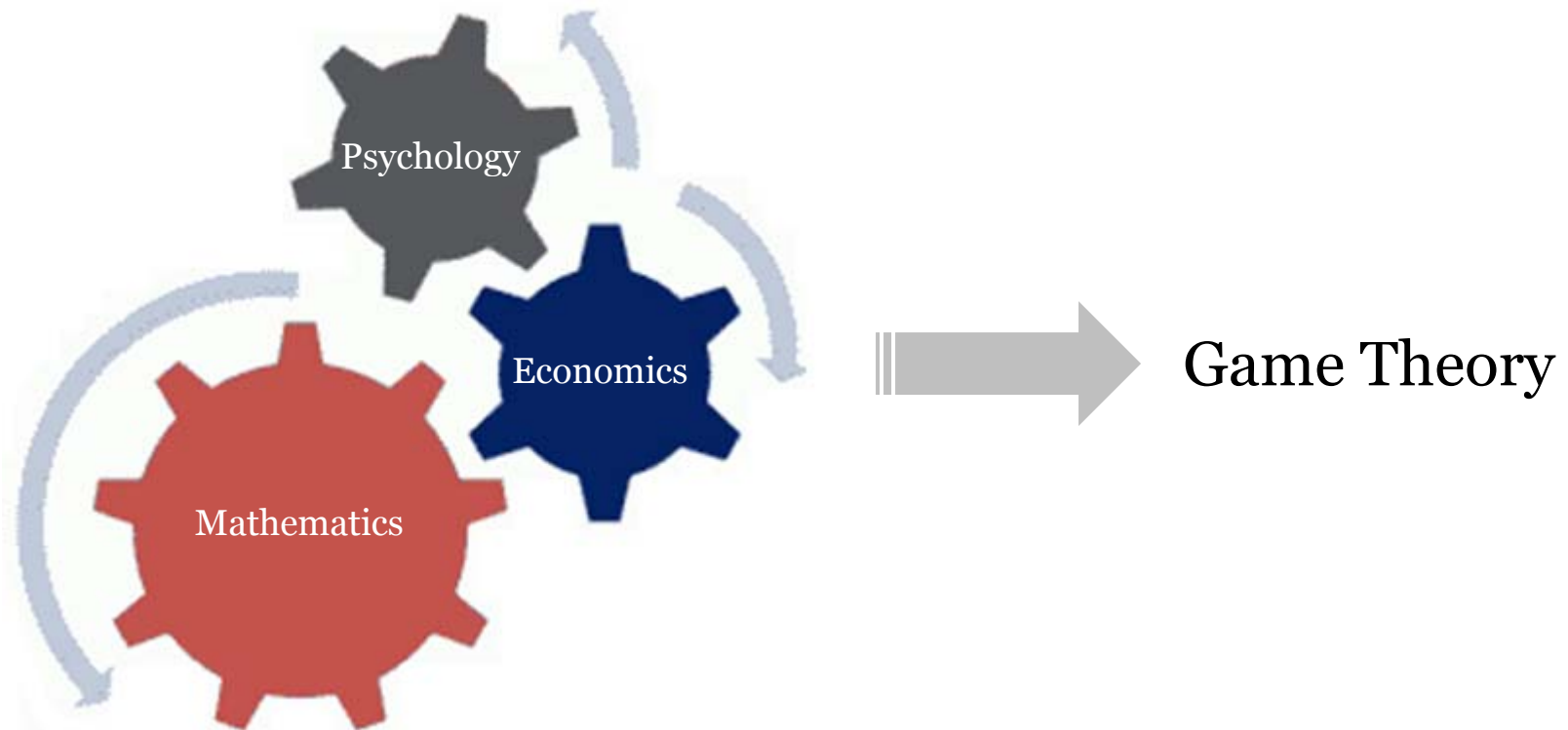
- Game Theory
- Types of Games
- Game Trees
- Minimax Search
- Alpha-Beta Pruning
- Summary

Outline

- Game Theory
- Types of Games
- Game Trees
- Minimax Search
- Alpha-Beta Pruning
- Summary

Game Theory

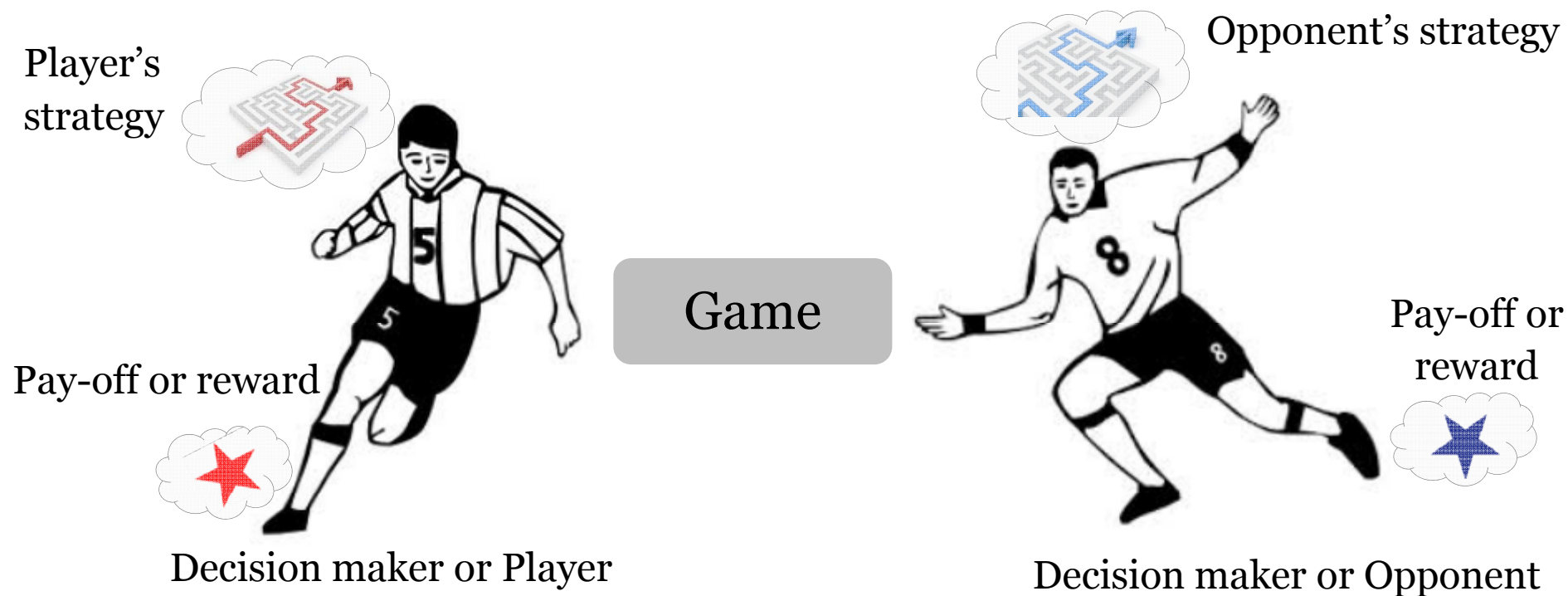
Game theory is an inter-disciplinary field that is devoted to the analysis of **interdependent decision making**.



Game Theory

Game theory is applicable to any social interaction in which:

- ◇ There are **two or more decision makers**, called **players**, each of whom faces a **choice between two or more courses of action**, called **strategies**;



Game Theory

- ◇ The **outcome of the interaction** depends on the **strategy choices** of all the players; and
- ◇ Each **player has preferences** among the possible outcomes, so that **a set of payoffs** reflecting these preferences can be assigned to **every outcome**.

Game Theory

Game Theory is the study of **decision making** in multi-person situations, where **the outcome depends on everyone's choice**.

The goal of each participant is to **maximize his/her own utility**, while taking into account that the **other participants are doing the same**.

Game Theory

- **Game Playing as Search**

- ◇ In Games there is an **opponent**.
- ◇ Need to **search for best move** and **wait for the opponent response to search again** from the new board configuration for best move.
- ◇ **Time is limited** to find goal in each search.

Game Theory

- **How to play a game?**
 - ◇ Consider all the **legal moves** you can make.
 - ◇ Each move leads to a **new board configuration** (position).
 - ◇ **Evaluate** each resulting position and determine which is best
 - ◇ **Make that move.**
 - ◇ **Wait** for your opponent to move and repeat?

Game Theory

- **How to play a game: Key problems**
 - ◇ Representing the “board”
 - ◇ Generating all legal next boards
 - ◇ Evaluating a position
 - ◇ Look ahead

Outline

- Game Theory
- **Types of Games**
- Game Trees
- Minimax Search
- Alpha-Beta Pruning
- Summary

Types of Games

Two-person Games

Strictly Competitive or zero-sum

The players' preferences are opposed: The winner's reward or payoff is exactly equal to the defeated player's loss.

Mixed-motive

The players' preferences among the possible outcomes coincide exactly

Pure Coordination

The players' preferences are neither strictly competitive nor strictly coincident, and which therefore possesses both competitive and cooperative features.

Prisoner's Dilemma Game

		II	
		C	D
I	C	3,3	1,4
	D	4,1	2,2

Chicken

		II	
		C	D
I	C	4,4	2,5
	D	5,2	1,1

[1]

Types of Games

- Nash Equilibrium – Prisoner's Dilemma



Prisoner B's Strategies



Prisoner A's
Strategies

		Prisoner B's Strategies	
		Cooperation	Confession
Prisoner A's Strategies	Cooperation	(3,3) (Reward for cooperation - both receive light sentences)	(0,5) (Sucker gets longer sentence if partner confesses)
	Confession	(5,0) (Temptation – reduced sentence for confessor)	(1,1) (Punishment for mutual confession – both receive intermediate sentences)

- Chicken Game



Driver B's Strategies



Driver A's
Strategies

		Driver B's Strategies	
		Swerve	Straight
Driver A's Strategies	Swerve	 	 
	Straight	 	

Types of Games

Games can be categorized based on:

- ◇ Whether or not **chance** can affect the **outcome of actions**,
- ◇ Whether or not all **state information** is available to all players.

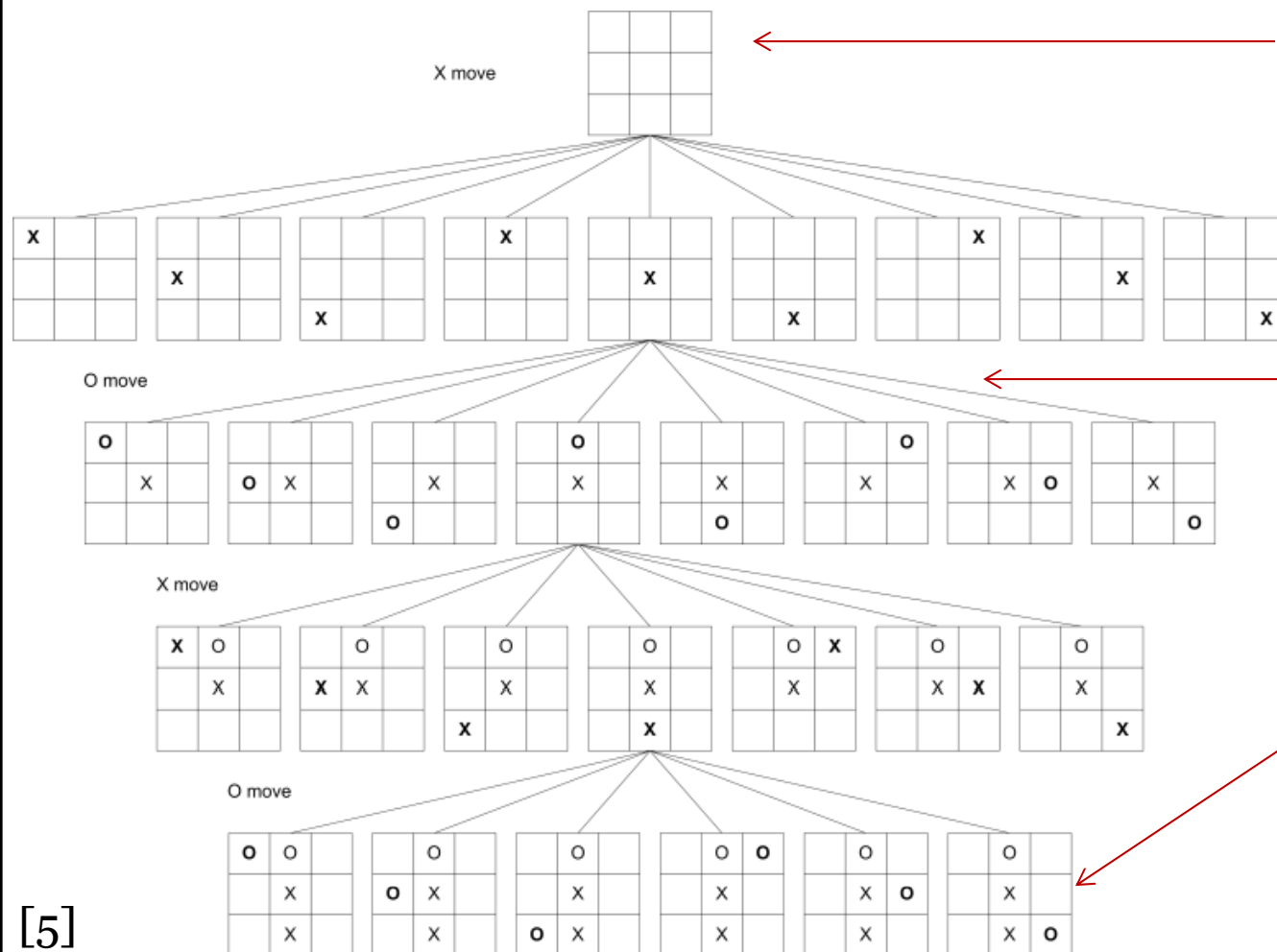
	Deterministic	Chance
Perfect information	Chess, checkers, go, othello	Backgammon, monopoly
Imperfect information	Battleship	Poker, Scabble, bridge

Outline

- Game Theory
- Types of Games
- **Game Trees**
- Minimax Search
- Alpha-Beta Pruning
- Summary

Game Trees

◇ Problem spaces for typical games represented as **trees**.



Root node represents the “**board**” configuration at which a decision must be made as to what is the best single move to make next.

Arcs represent the **possible legal moves** for a player (no costs associates to arcs)

Terminal nodes represent end-game configurations (the result must be one of “win”, “lose”, and “draw”, possibly with numerical payoff) .

[5]

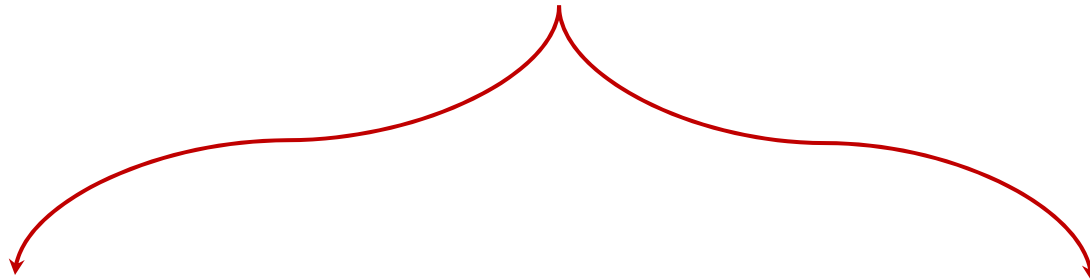
Game Trees

- ◇ Problem spaces for typical games represented as **trees**.
- ◇ **Root node** represents the “**board**” configuration at which a decision must be made as to what is the best single move to make next. (not necessarily the initial configuration)
- ◇ **Evaluator function** rates a board position. $f(\text{board})$ (a real number).
- ◇ **Arcs** represent the **possible legal moves** for a player (no costs associates to arcs)
- ◇ **Terminal nodes** represent end-game configurations (the result must be one of “win”, “lose”, and “draw”, possibly with numerical payoff) .

[4]

Game Trees

Game Tree



Complete game tree

includes all configurations that can be generated from the root by legal moves (all leaves are terminal nodes)

Incomplete game tree

includes all configurations that can be generated from the root by legal moves to a given depth (looking ahead to a given steps or trails)

Game Trees

- **Evaluation function or static evaluator**
 - ◇ Evaluates the “**goodness**” of a game position.
 - ◇ Assumed to be **better** estimate as **search is deepened** (i.e., at lower levels of game tree).
 - ◇ Contrast with **heuristic search** where the evaluation function was a non-negative estimate of the **cost** from the start node to a goal and passing through the given node.

Game Trees

- **Evaluation function or static evaluator**

- ◇ The **zero-sum assumption** allows us to use a single evaluation function to describe the **goodness** of a board **with respect to both players**.

- $f(n) > 0$: position n good for me and bad for you.
 - $f(n) < 0$: position n bad for me and good for you
 - $f(n)$ near 0: position n is a neutral position.
 - $f(n) \gg 0$: win for me.
 - $f(n) \ll 0$: win for you.

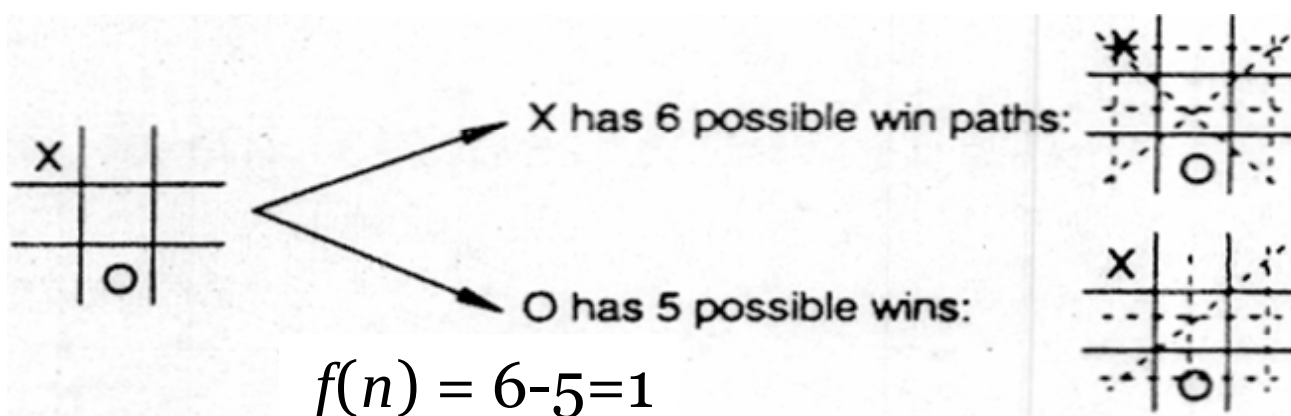
Game Trees

- **Evaluation function: Tic-Tac-Toe**

◇ $f(n) = [\text{\# of 3-lengths open for me}] - [\text{\# of 3-lengths open for you}]$

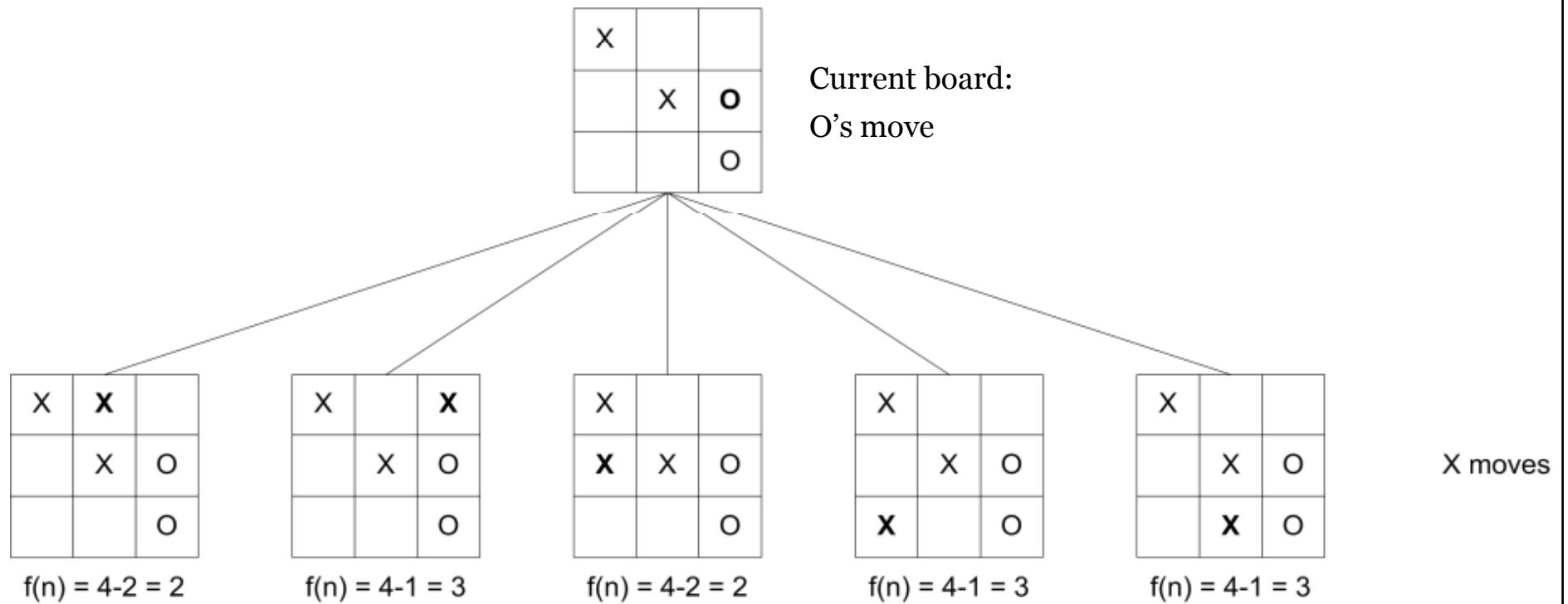


where a 3-length is a complete row, column, or diagonal or in other words number of **possible wins** (rows, columns, diagonals) **not blocked** by opponent, minus number of possible wins for opponent not blocked by current player



Game Trees

- **Evaluation function: Tic-Tac-Toe**



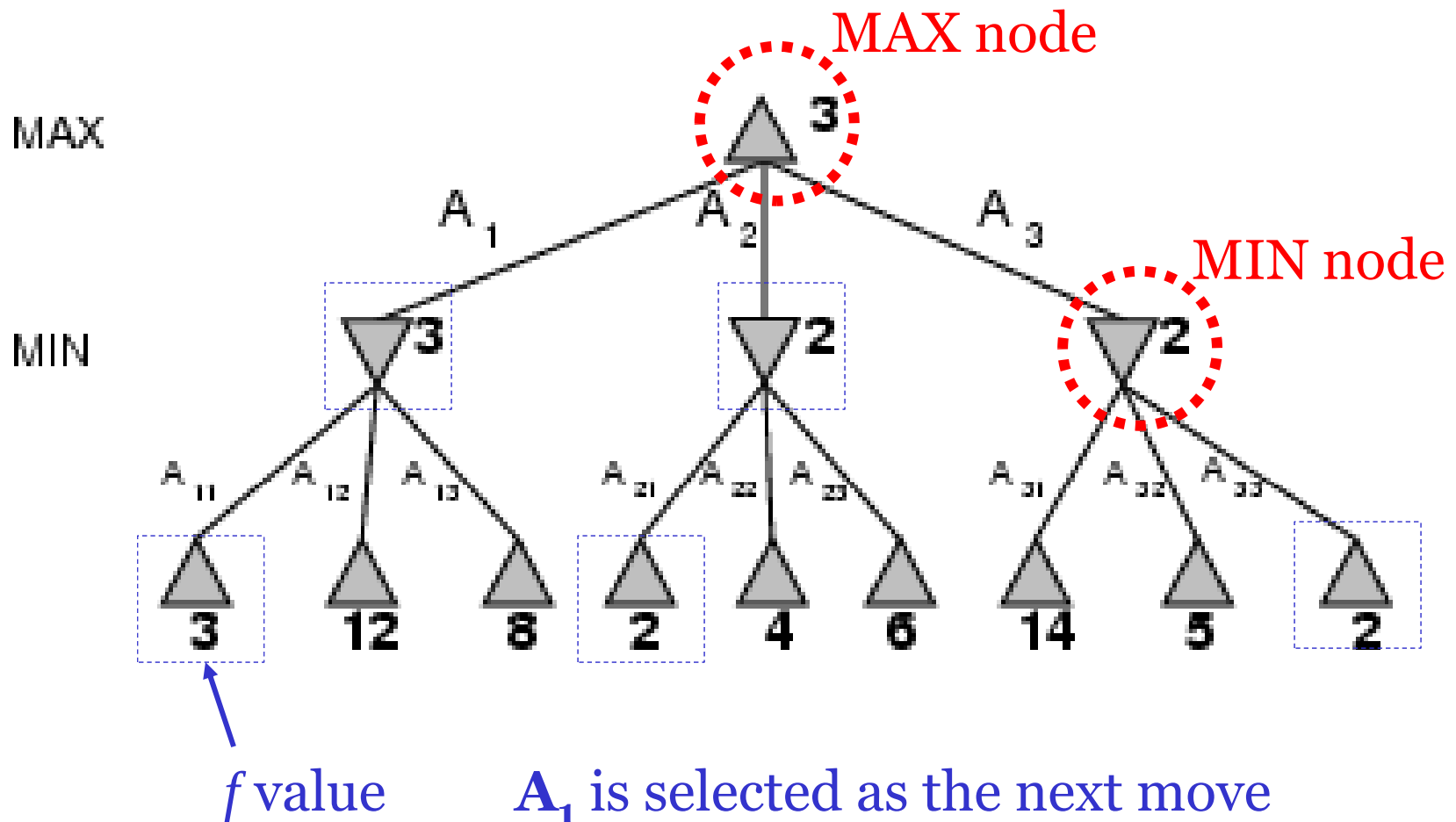
Outline

- Game Theory
- Types of Games
- Game Trees
- **Minimax Search**
- Alpha-Beta Pruning
- Summary

Minimax Search

- **Goal of game tree search:** to determine one move for **MAX (player)** that **maximizes** the guaranteed **payoff** for a given game tree for MAX regardless of the moves the **MIN (opponent)** will take.
- The value of each node (MAX and MIN) is determined by (**back up from**) the values of **its children**.
- For a **MAX node**, the backed up value is the **maximum of the values associated with its children**.
- For a **MIN node**, the backed up value is the **minimum of the values associated with its children**.

Minimax Search



Minimax Search

- **Minimax principle**

- ◇ Label each level in Game Tree with **MAX (player)** and **MIN (opponent)**
- ◇ Label **leaves** with **evaluation** of player.
- ◇ Go through the game tree
 - if **father node** is **MAX** then label the node with the **maximal value** of its successors.
 - if **father node** is **MIN** then label the node with the **minimal value** of its successors.

Minimax Search

- Minimax principle

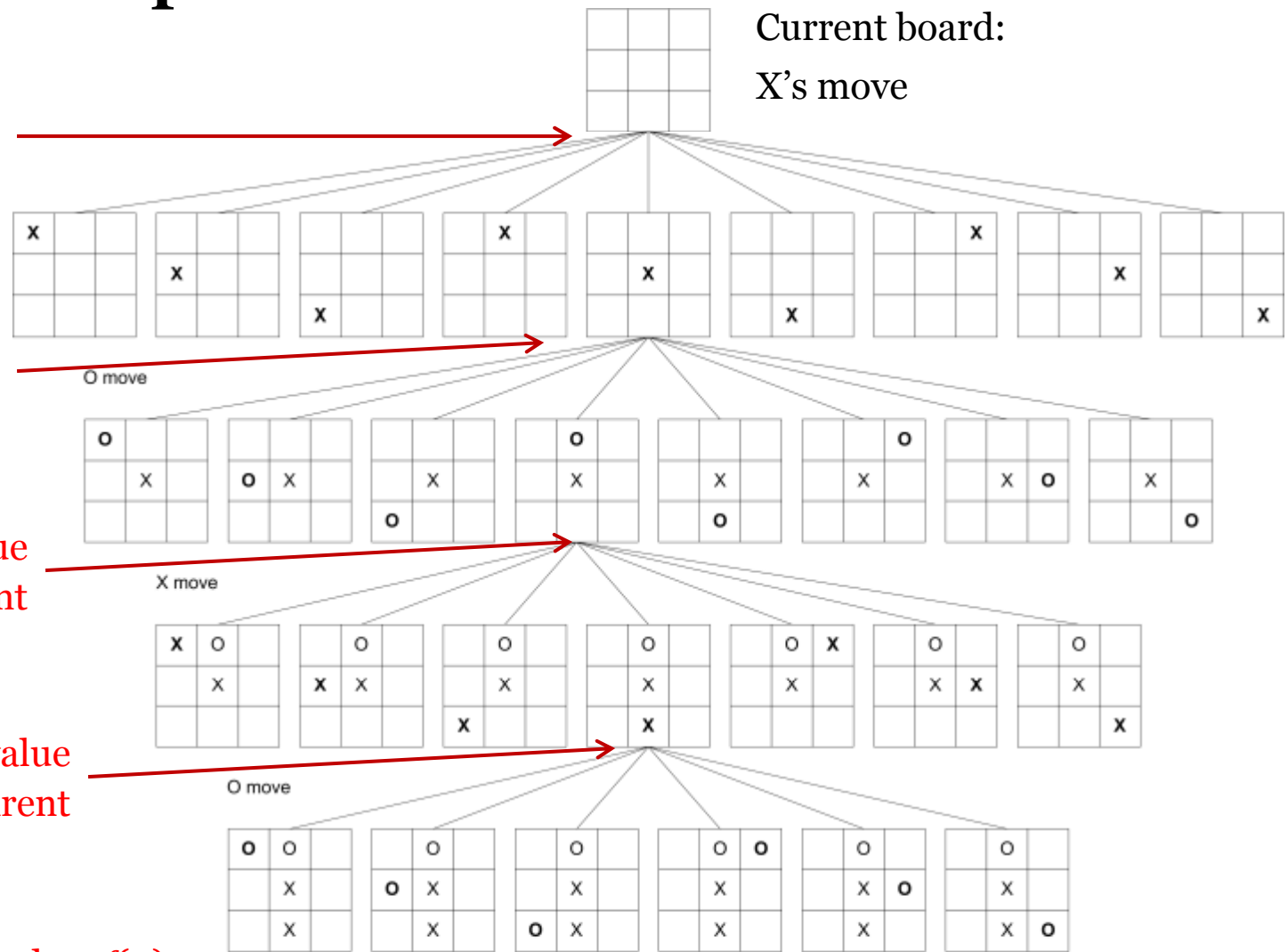
Propagate max value
of children to parent

Propagate min value
of children to parent

Propagate max value
of children to parent

Propagate min value
of children to parent

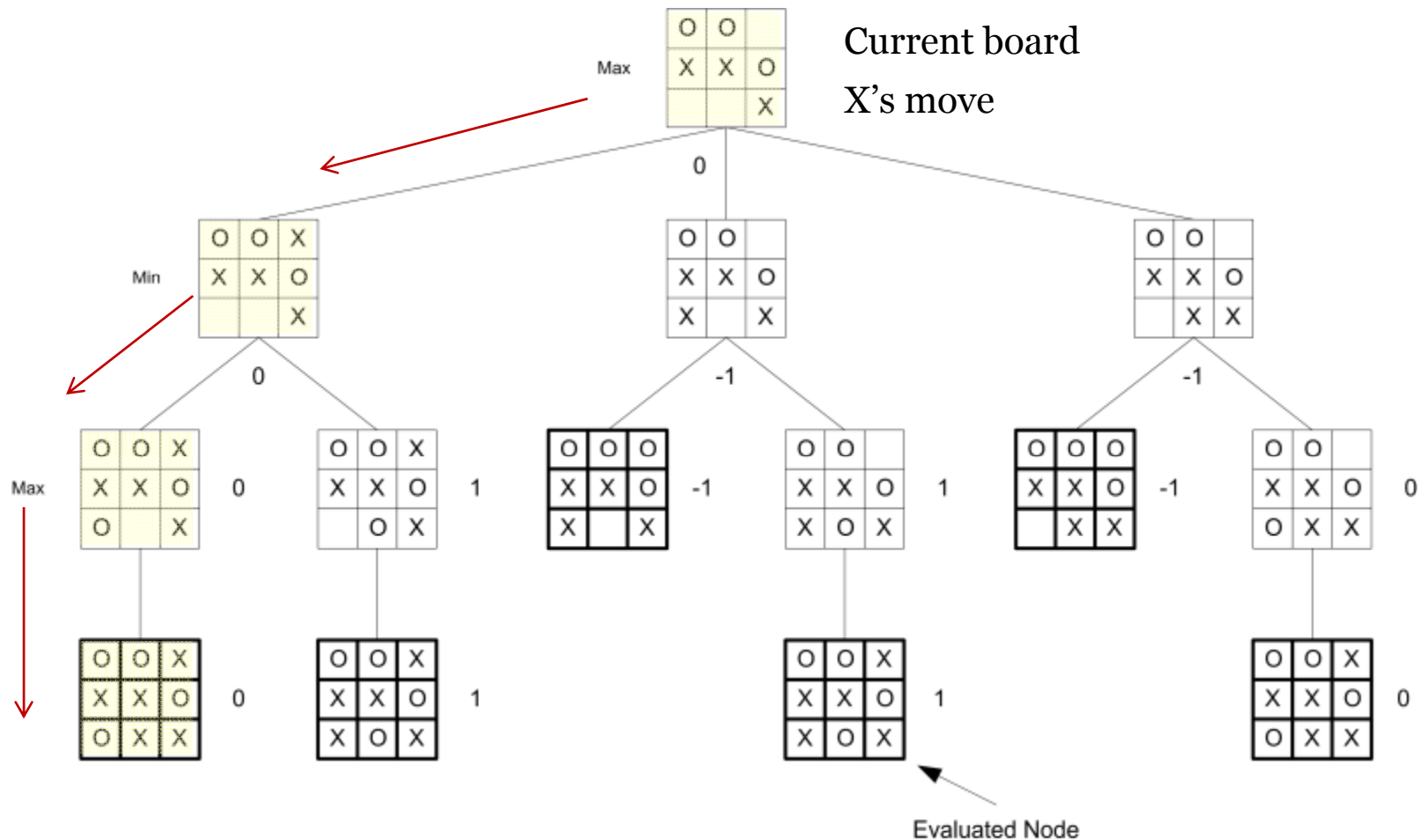
Calculate $f(n)$



[6]

Minimax Search

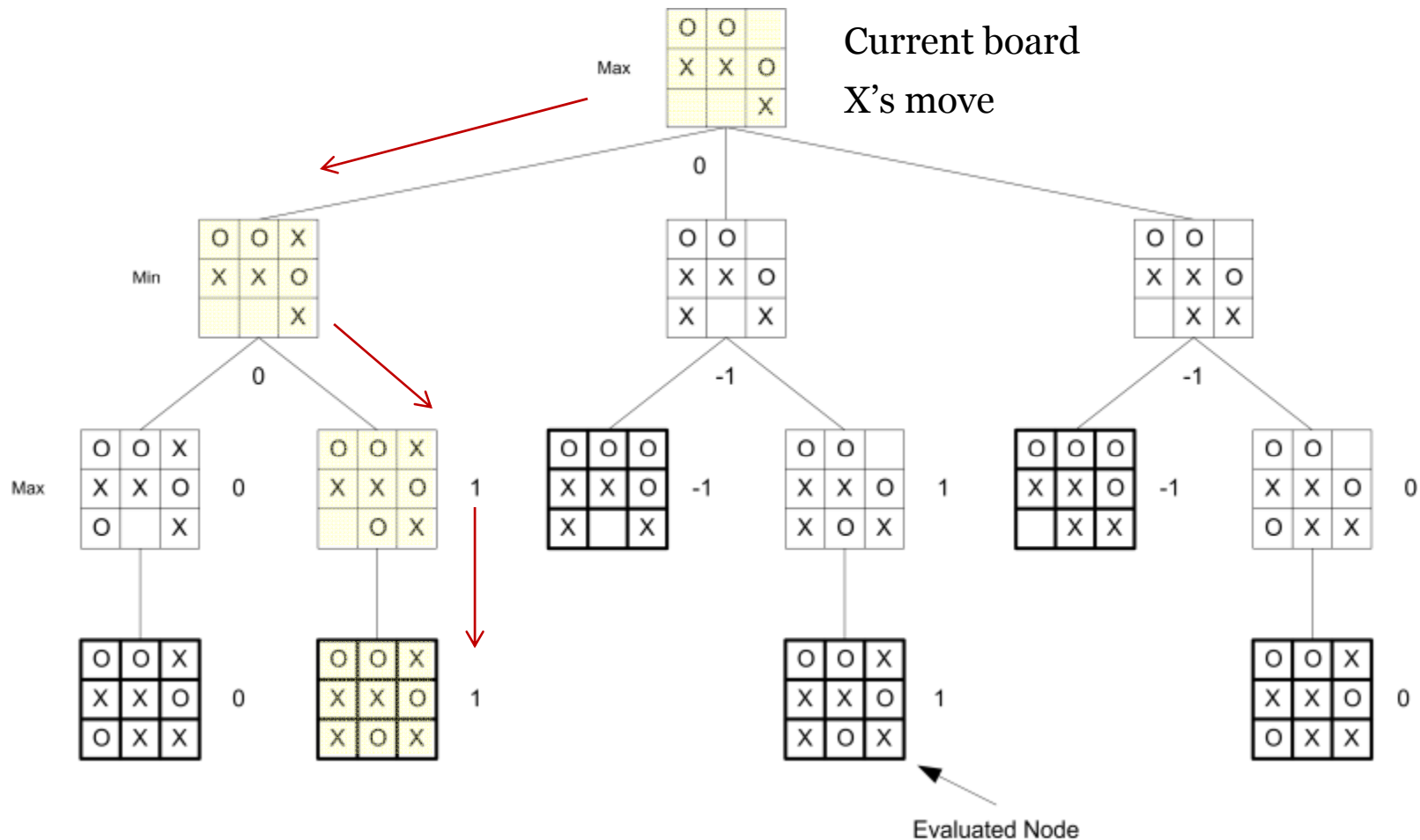
- Minimax principle



Following minimax strategy leads to draw or Tie Game

Minimax Search

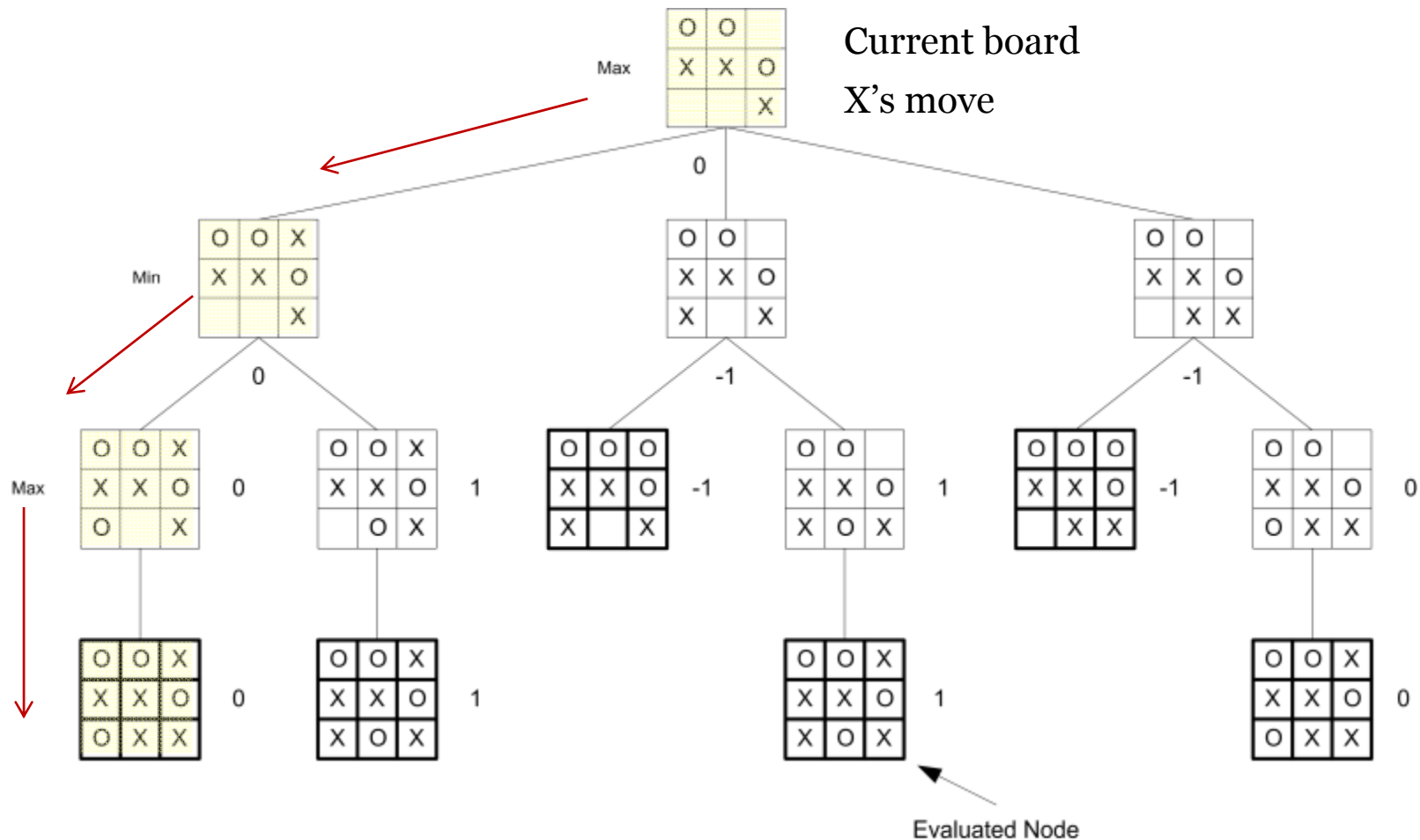
- Minimax principle



Not following minimax strategy leads to win

Minimax Search

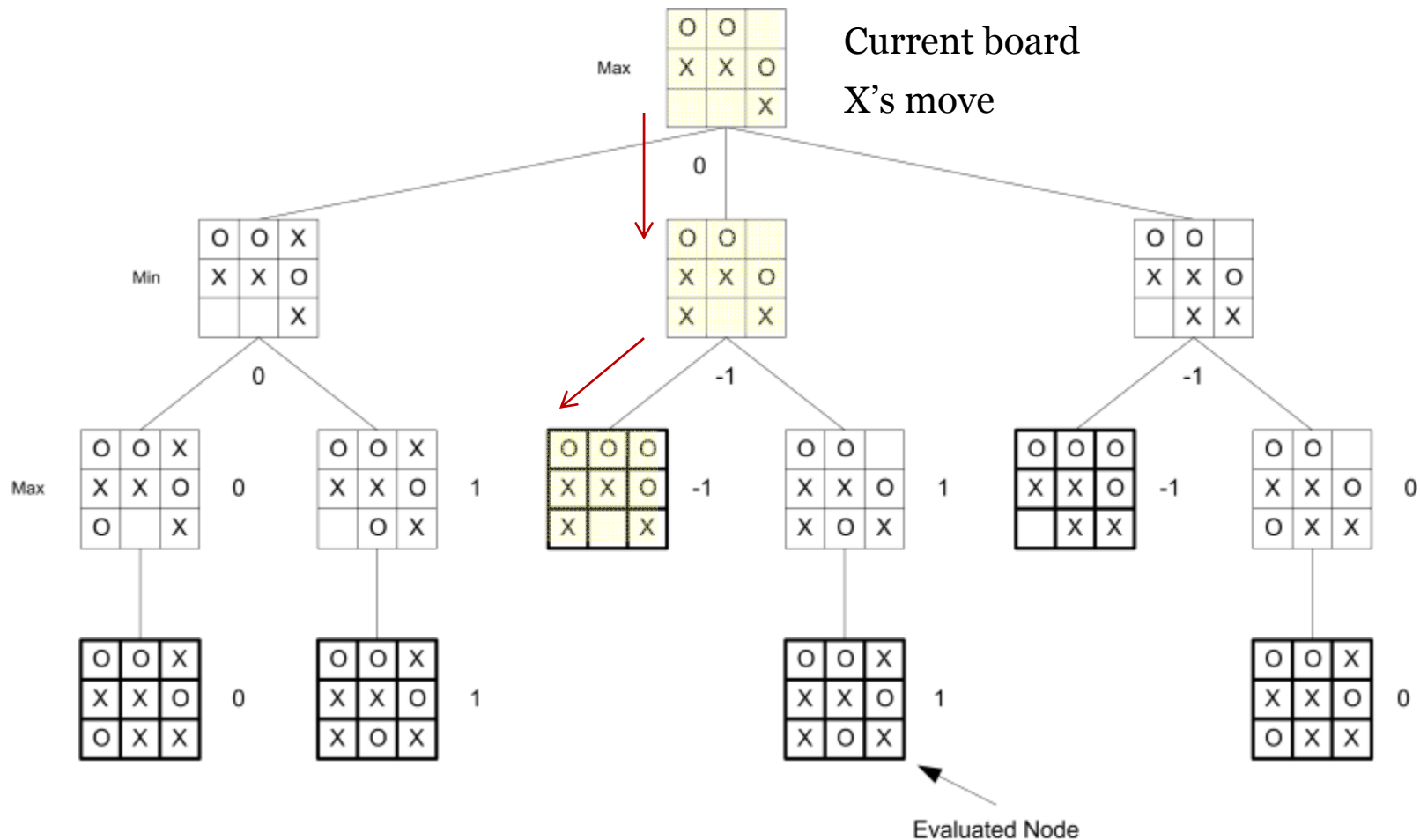
- Minimax principle



Not following minimax strategy leads to draw

Minimax Search

- Minimax principle



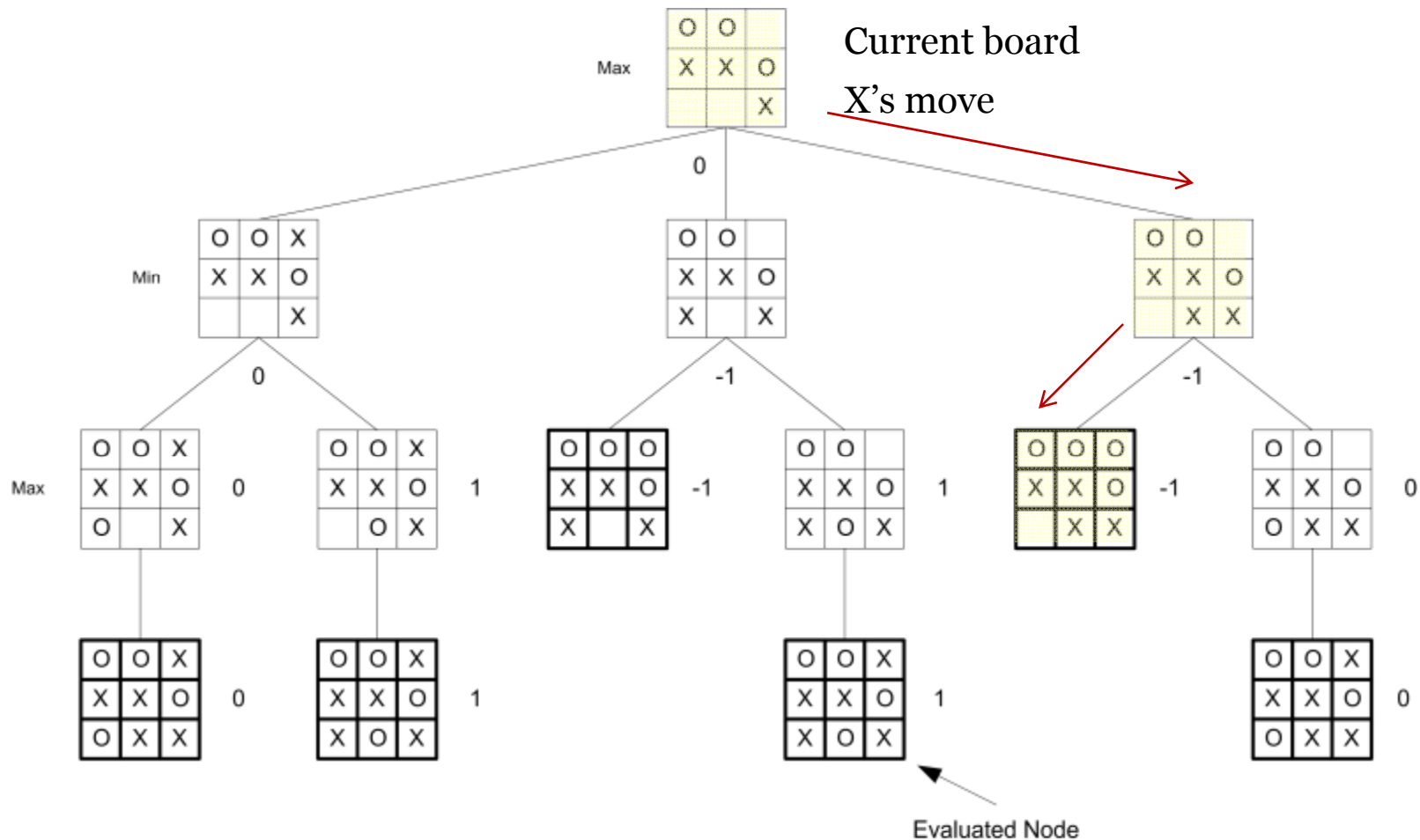
Not following minimax strategy leads to lose

- **Minimax principle**



Minimax Search

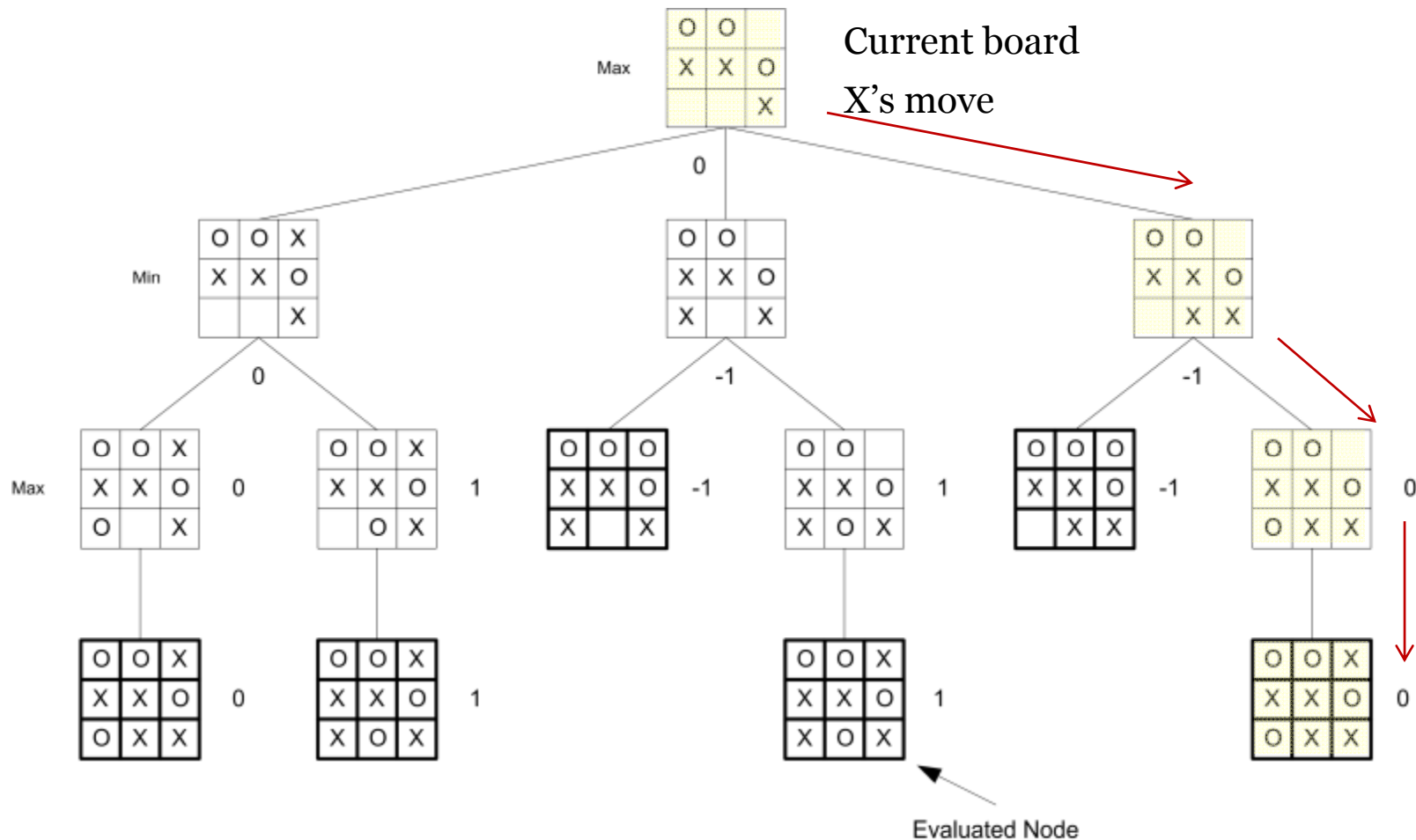
- Minimax principle



Not following minimax strategy leads to lose

Minimax Search

- Minimax principle

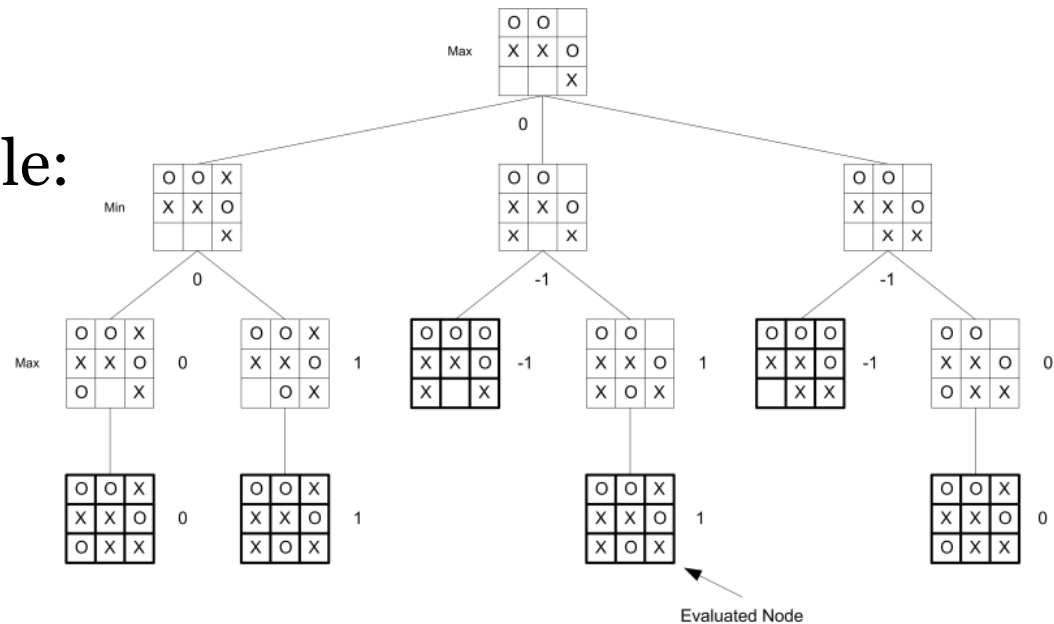


Not following minimax strategy leads to draw or tie game

Minimax Search

- Minimax principle**

For this specific example:



Case	Probability
Following minimax strategy	Draw with probability $P=1$
Without following minimax	- Draw with $P=2/6=1/3$ - Win with $P=2/6=1/3$ - Lose with $P=2/6=1/3$

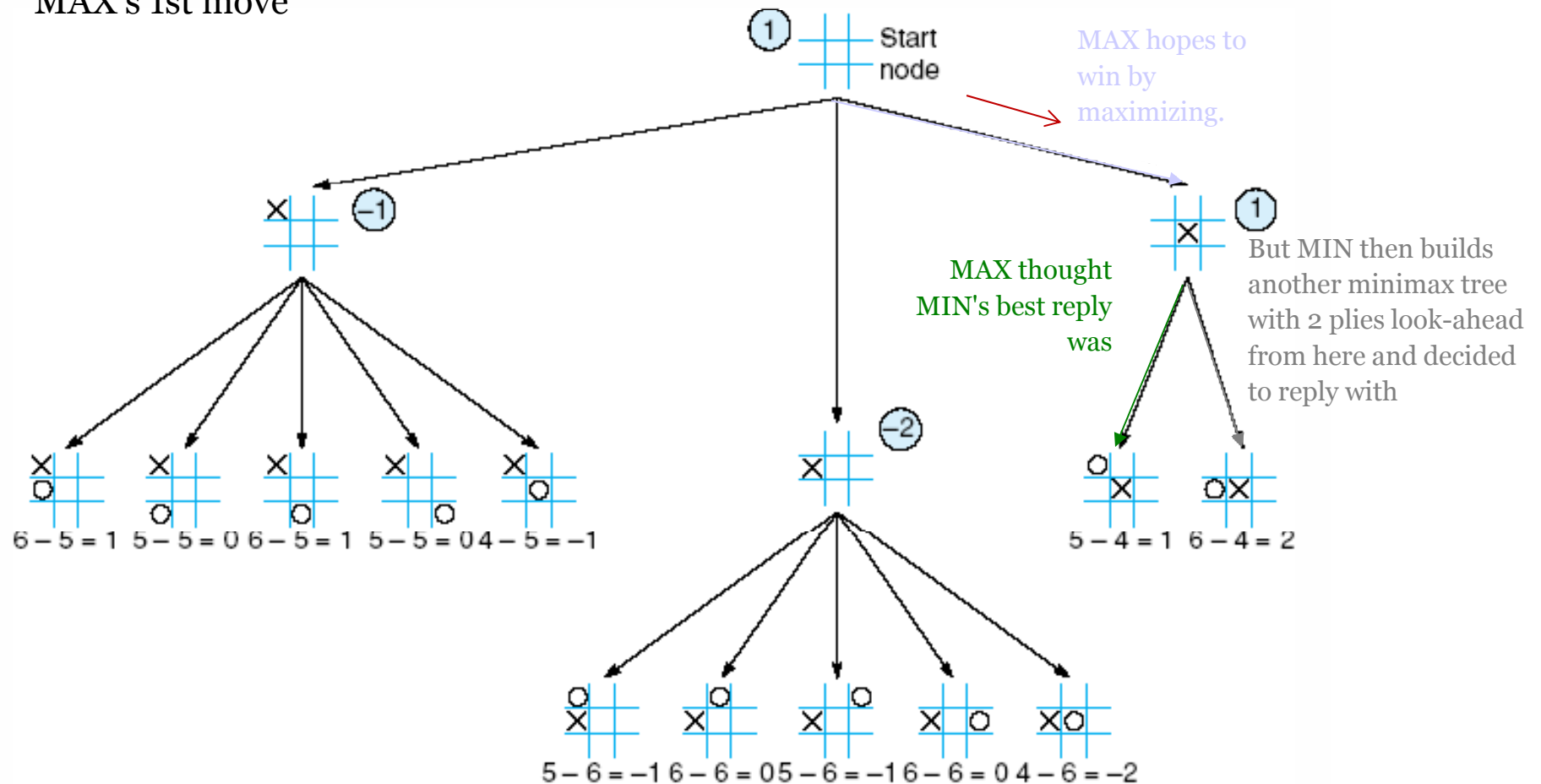
Moreover, computer games should follow well-defined search strategy. Minimax rule is a basis for other game tree search algorithms

Minimax Search

- Minimax principle**

Two-ply minimax applied to opening move of tic-tac-toe

MAX's 1st move

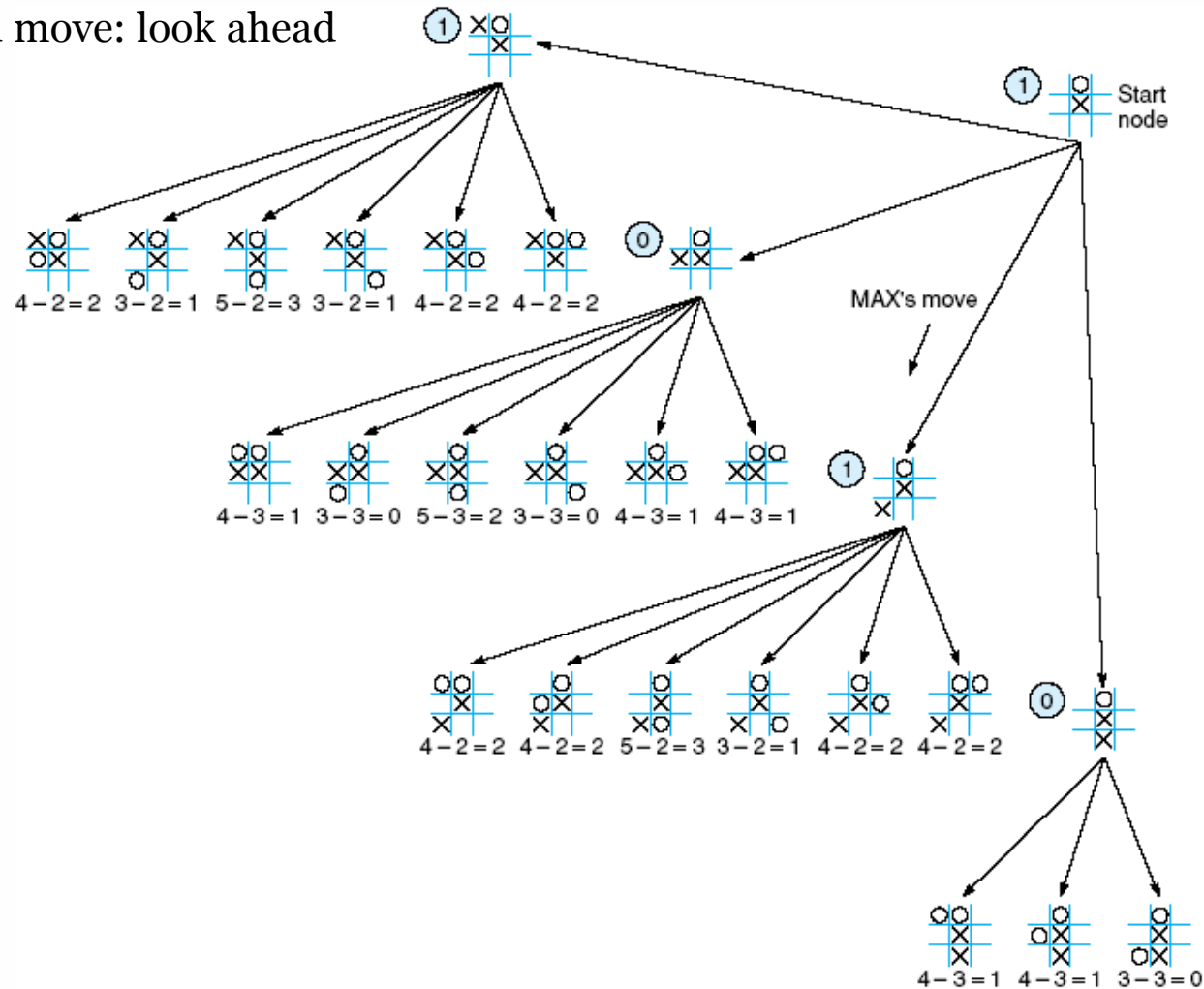


Minimax Search

- Minimax principle**

Two-ply minimax applied to X's second move of tic-tac-toe.

MAX's 2nd move: look ahead analysis



Minimax Search

- **Minimax principle**

- ◇ Minimax rule is a basis for other game tree search algorithms.
- ◇ Complete: Yes (if tree is finite)
- ◇ Time complexity: **$O(b^d)$**
- ◇ Space complexity: **$O(bd)$** (**depth-first** exploration)
- ◇ Optimal: Yes (against an optimal opponent)

Minimax Search

- **Minimax principle**

- ◇ Expanding the complete game tree is only feasible for simple games.
- ◇ Tic Tac Toe has average **branching factor b** of 5 and average (moves) **depth d** of 9.
- ◇ **Checkers** has $b = 35$ and $d = 100$. Taking into consideration symmetrical and repeated states the game tree may be less than that. Still for checkers **1040**. For **Chess** it is **10^{140}** .
- ◇ It is only feasible to explore a limited depth of the game tree and to use a board evaluation function for the worth of this node to the player.

[3]

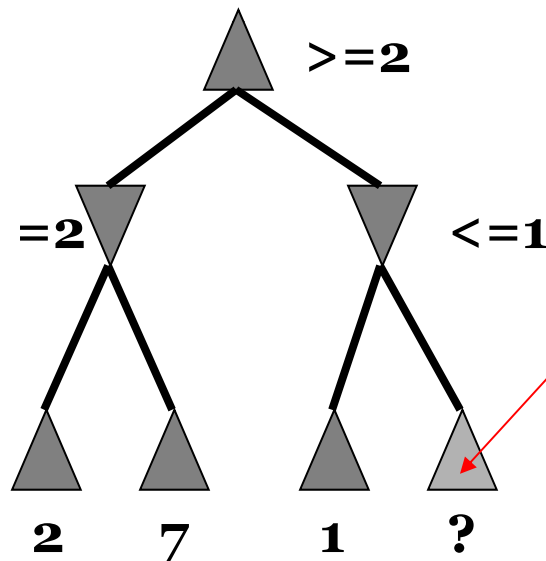
Outline

- Game Theory
- Types of Games
- Game Trees
- Minimax Search
- **Alpha-Beta Pruning**
- Summary

Alpha-Beta Pruning

- We can improve on the performance of the minimax algorithm through **alpha-beta pruning**.
- **Basic idea:** *“If you have an idea that is surely bad, don’t take the time to see how truly awful it is.”* -- Pat Winston

American computer scientist

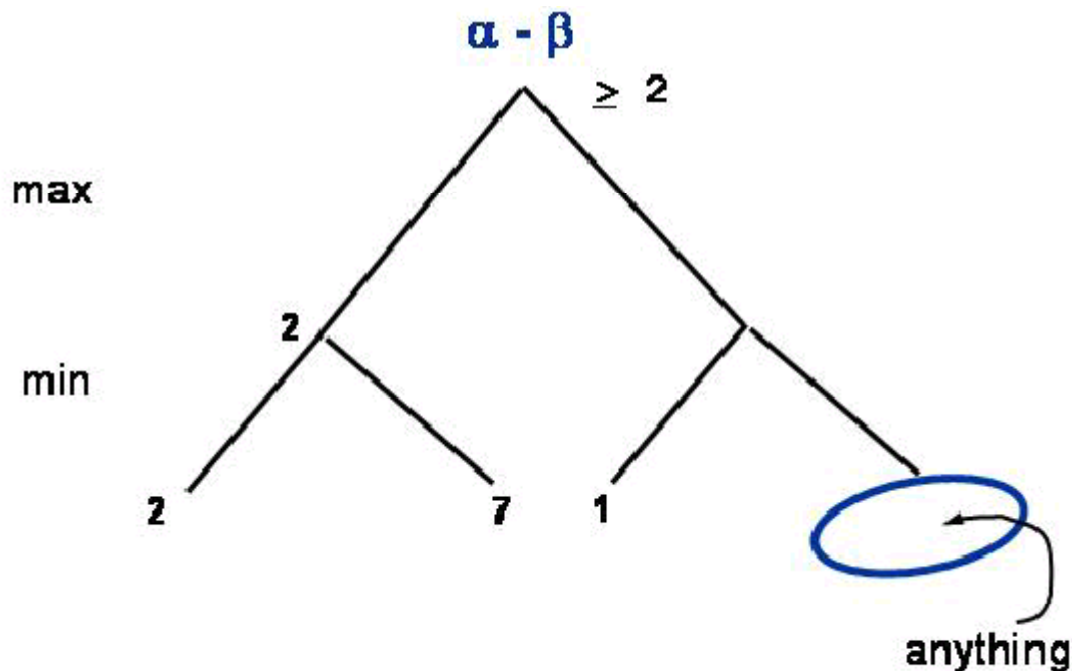


- We don't need to compute the value at this node.
- No matter what it is it can't effect the value of the root node.

- A way to **reduce the number of nodes** that need to generated and evaluated.

[4]

Alpha-Beta Pruning



- α -values at Max nodes **never decrease**.
- β -values at Min nodes **never increase**.

Alpha-Beta Pruning

- Traverse the search tree in **depth-first order**
- At each **Max node n** , **$\alpha(n)$** = **maximum value** found so far:
 - ◇ Start with $-\infty$ and only **increase**
 - ◇ Increases if a child of n returns a value **greater than** the current α
 - ◇ Serve as a tentative **lower bound** of the **final pay-off**

Alpha-Beta Pruning

- At each **Min node n**, **beta(n)** = **minimum value** found so far:
 - ◇ Start with ∞ and only **decrease**
 - ◇ Decreases if a child of n returns a value **less than** the current beta
 - ◇ Serve as a tentative **upper bound** of the **final pay-off**

Alpha-Beta Pruning

- **Alpha cutoff:** when the value of a **min position** is **less than or equal to the alpha-value** of its parent, **stop generating** further successors.
- Given a **Max node n** , cutoff the search below **n** (i.e., don't generate or examine any more of n 's children)

if **$\alpha(n) \geq \beta(n)$**

(alpha increases and passes beta from below)

Alpha-Beta Pruning

- **Beta cutoff:** when the value of a **max position** is **greater than the beta-value** of its parent, **stop generating** further successors.
- Given a **Min node n** , cutoff the search below **n** (i.e., don't generate or examine any more of n 's children)

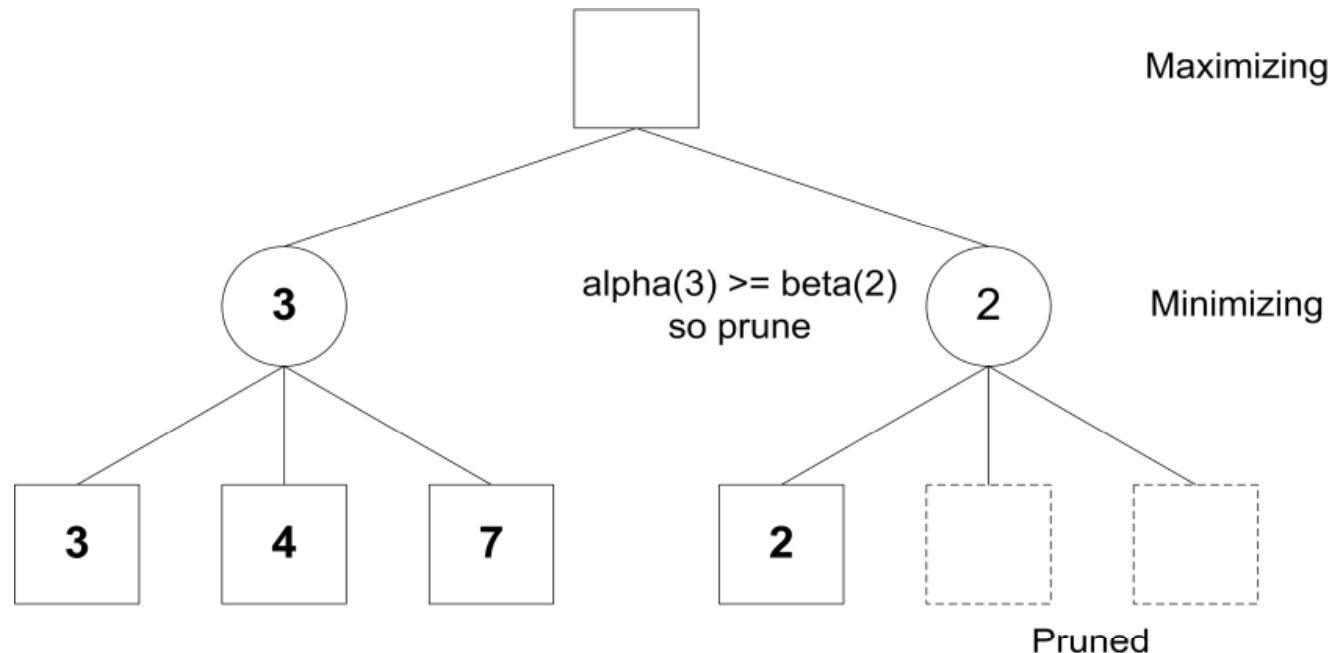
if **$\text{beta}(n) \leq \text{alpha}(n)$**

(beta decreases and passes alpha from above)

- Carry alpha and beta values down during search

Pruning occurs whenever **$\text{alpha} \geq \text{beta}$**

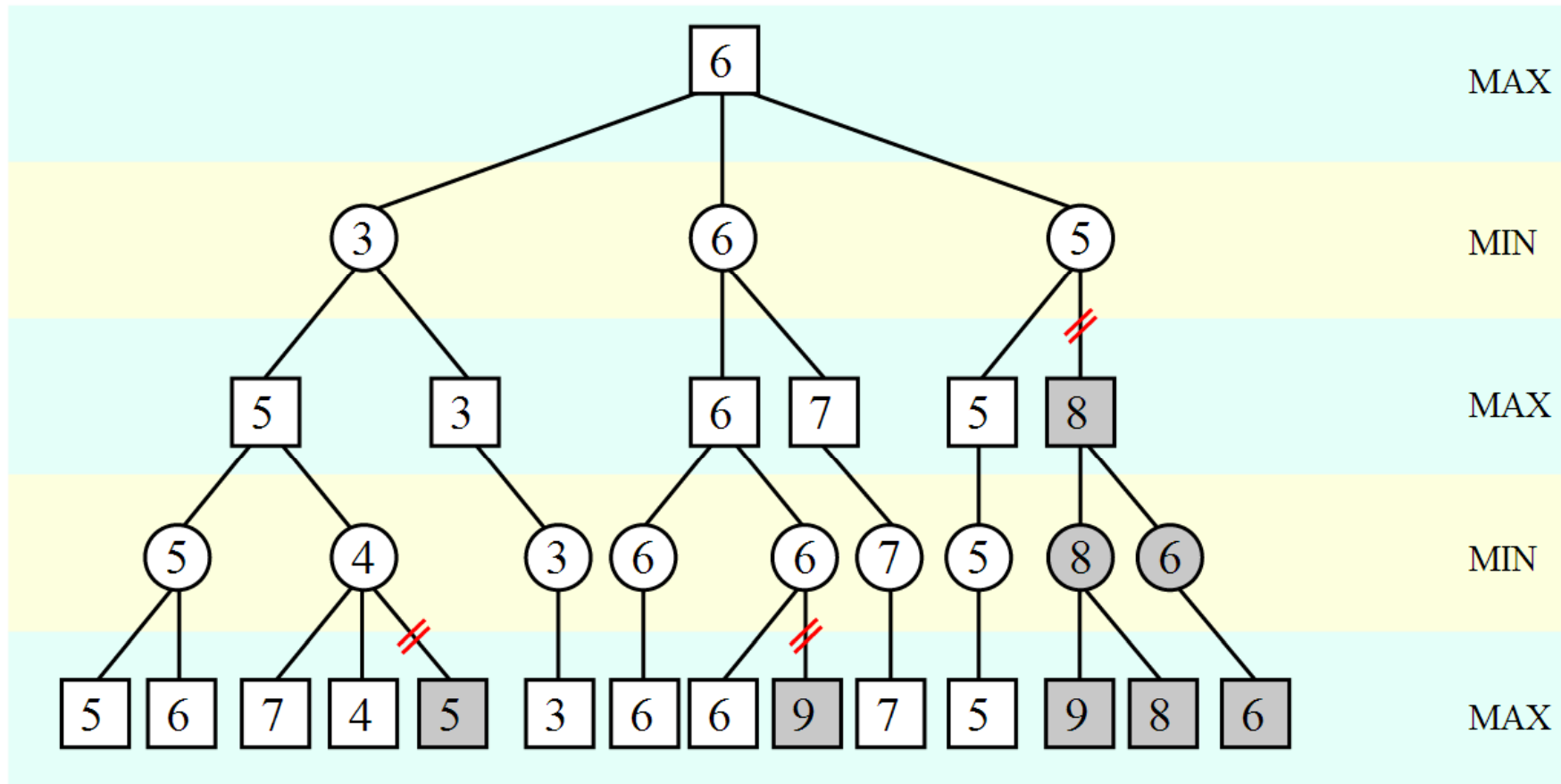
Alpha-Beta Pruning



- α = value of the **best possible move you can make**, that you have computed so far
- β = value of the **best possible move your opponent** can make, that you have computed so far
- If at any time, $\alpha \geq \beta$, then your opponent's best move can force a worse position than your best move so far, and so there is **no need to further evaluate this move**.

Alpha-Beta Pruning

- Example



Source: http://en.wikipedia.org/wiki/Alpha%20%80%93beta_pruning

Alpha-Beta Pruning

- **Algorithm**

Initialization: $\alpha = -\infty$ and $\beta = \infty$

function MAX-value (n, alpha, beta)

if n is a leaf node **then return** f(n);

for each child n' of n **do**

 alpha := max{alpha, MIN-value(n', alpha, beta)};

if alpha $\geq$ beta **then return** beta /* pruning */

end{do}

return alpha

function MIN-value (n, alpha, beta)

if n is a leaf node **then return** f(n);

for each child n' of n **do**

 beta := min{beta, MAX-value(n', alpha, beta)};

if beta $\leq$ alpha **then return** alpha /* pruning */

end{do}

return beta

[4]

Alpha-Beta Pruning

$$\min(A) = -1$$

$$\max(S) \text{ must } \geq -1$$

Lower bound: α -value of S = -1

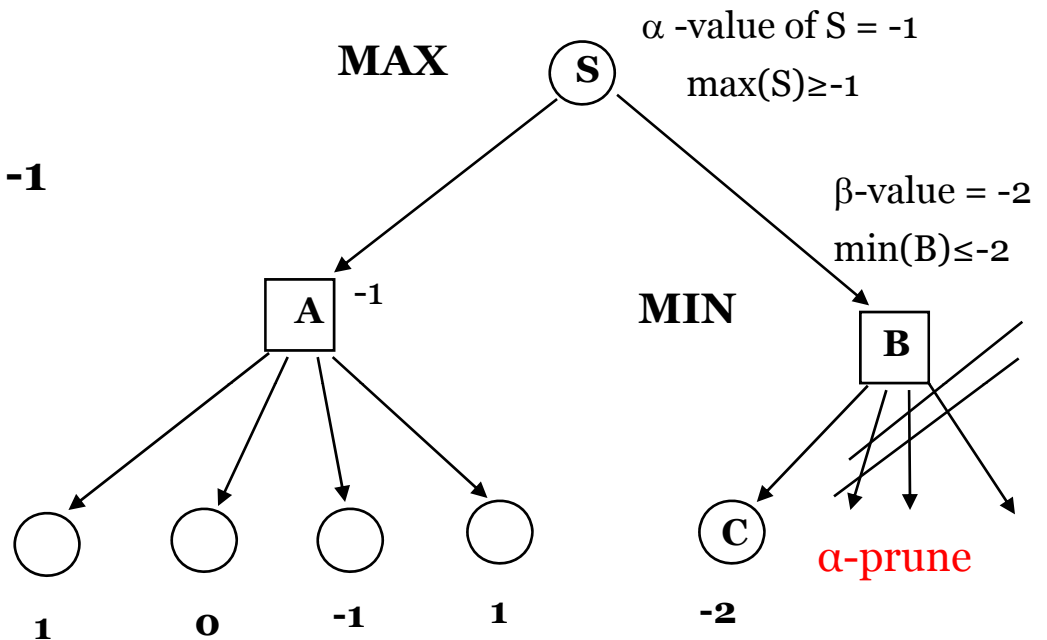
$$f(C) = -2$$

$$\min(B) \leq -2$$

Upper bound: β -value = -2

$$\alpha\text{-value} > \beta\text{-value}$$

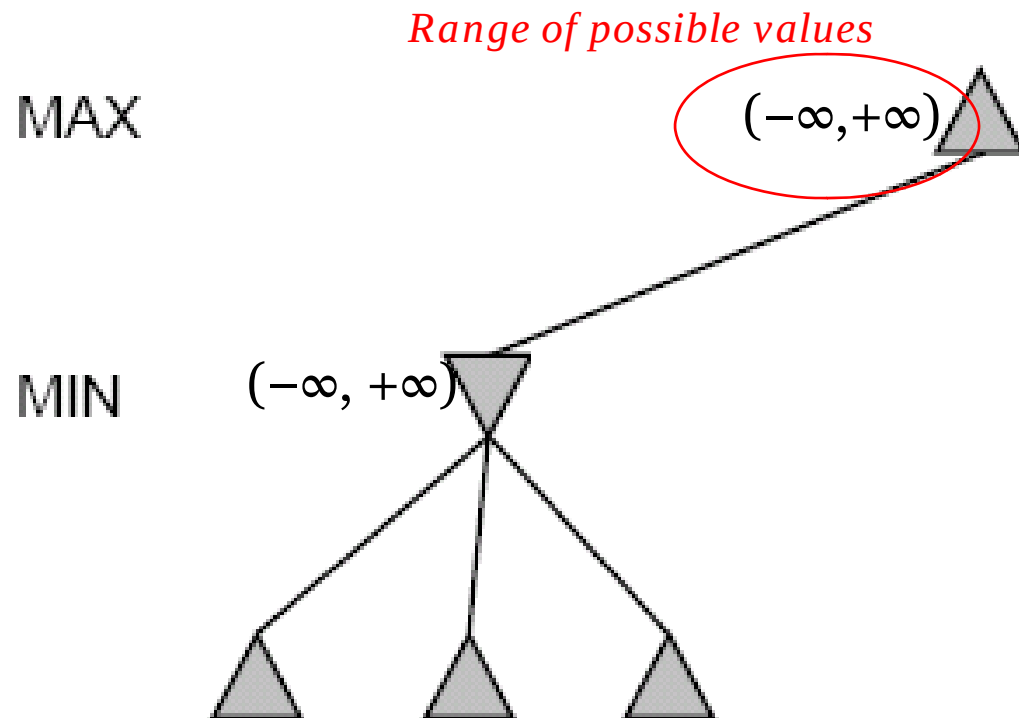
Final value of B can never exceed current value of S. We can prune the other children of B.



Alpha-Beta Pruning

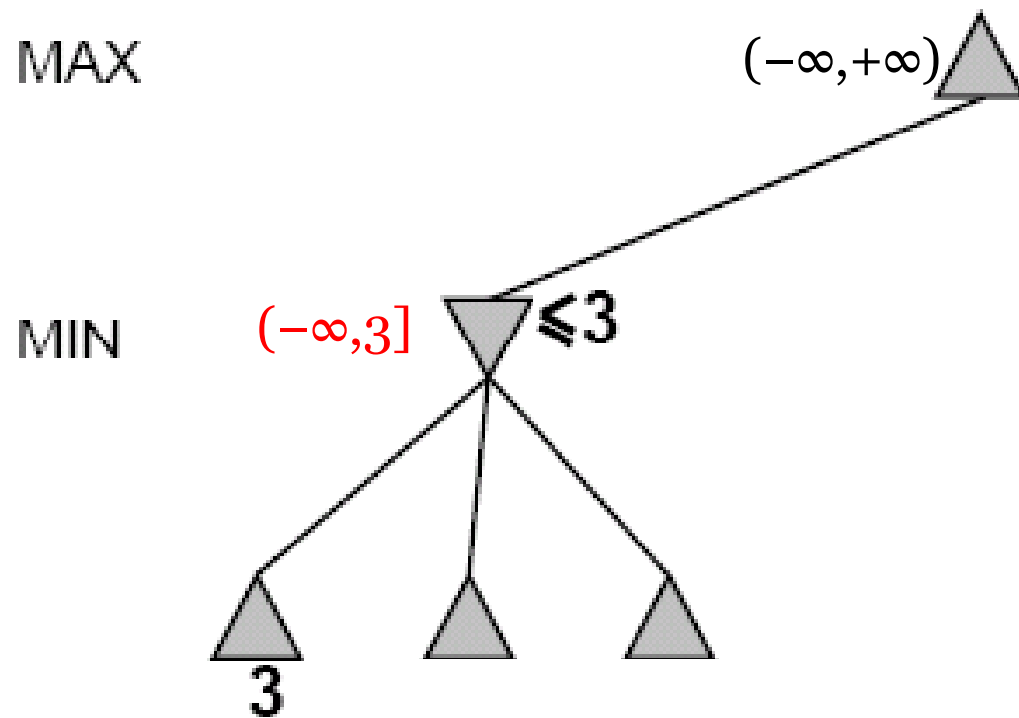
- Example

Do DF-search until first leaf



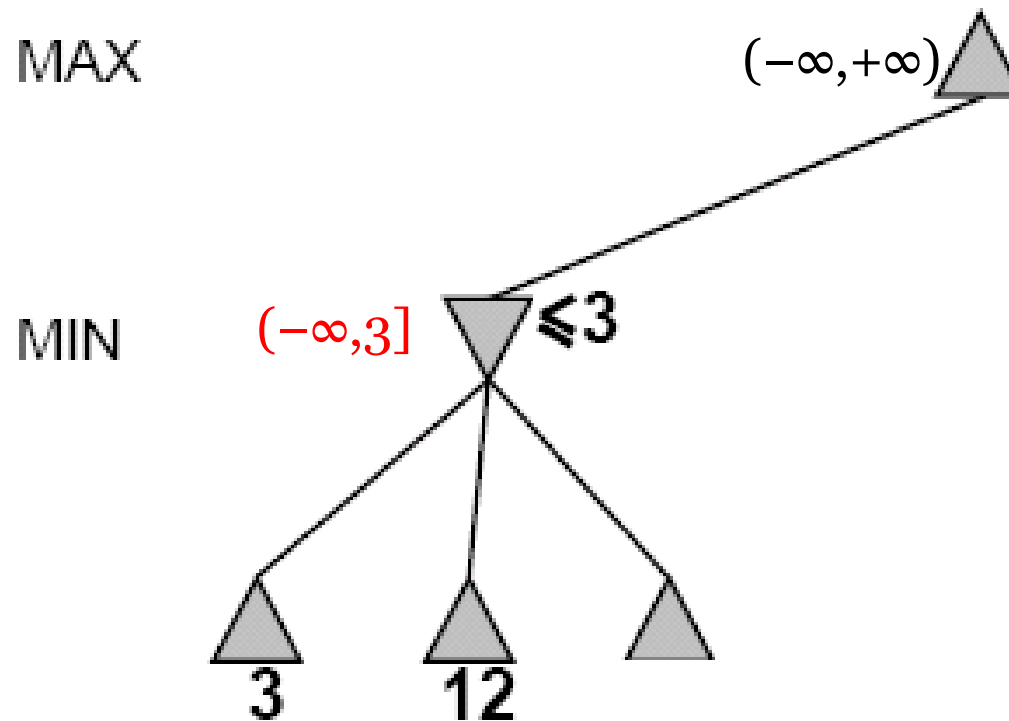
Alpha-Beta Pruning

- Example



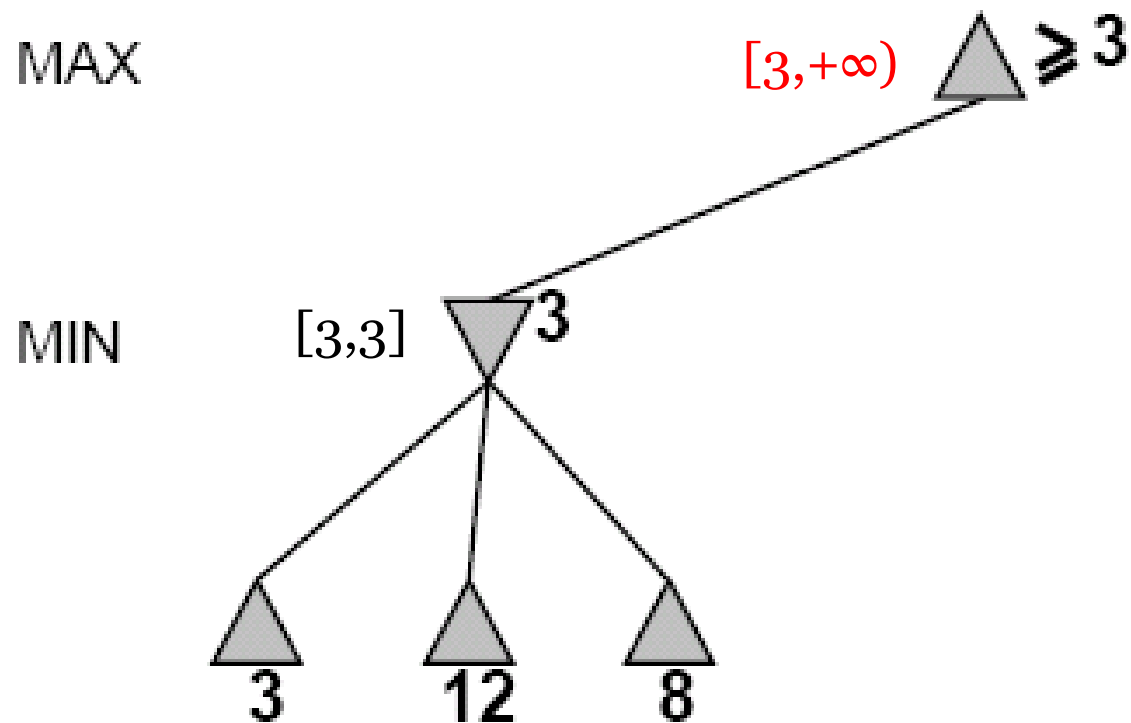
Alpha-Beta Pruning

- Example



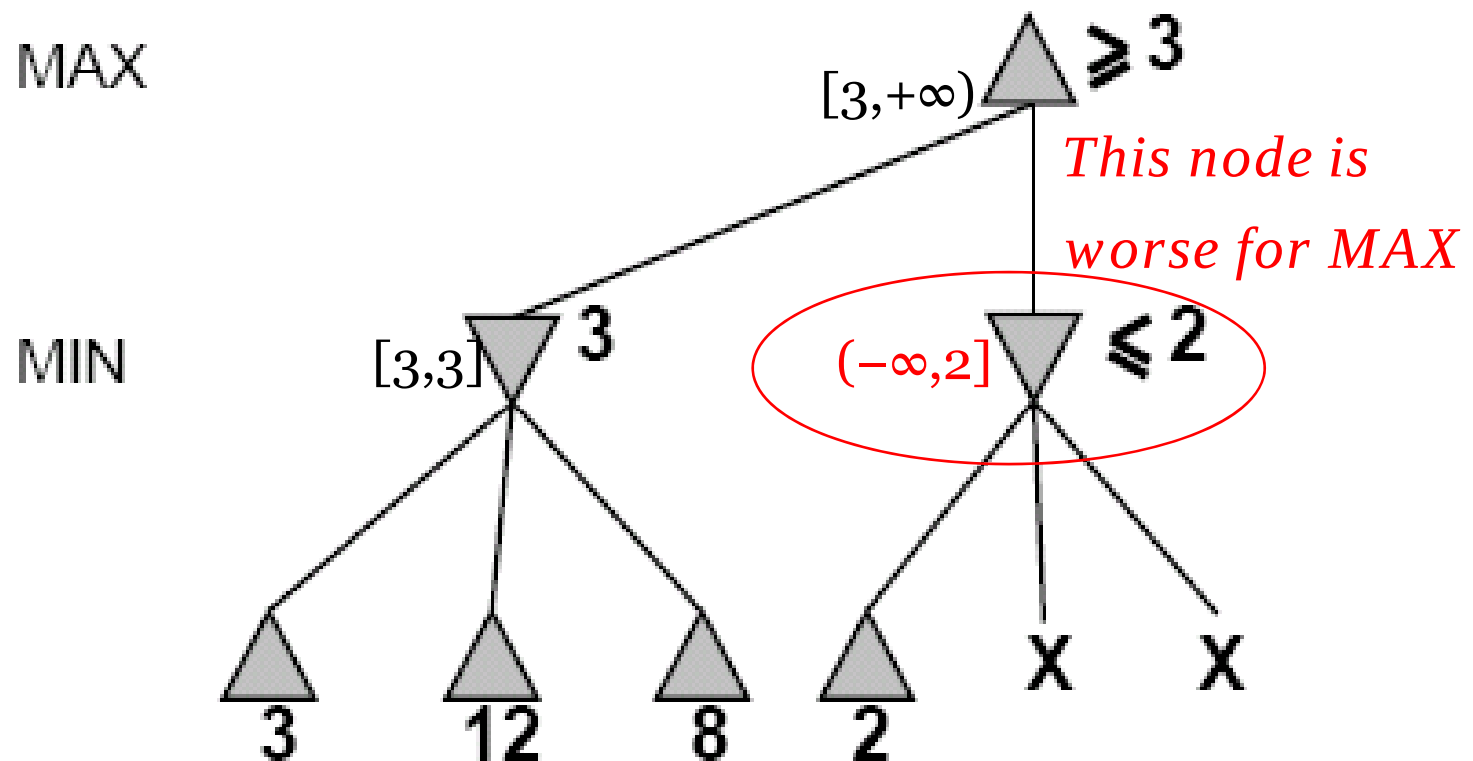
Alpha-Beta Pruning

- Example



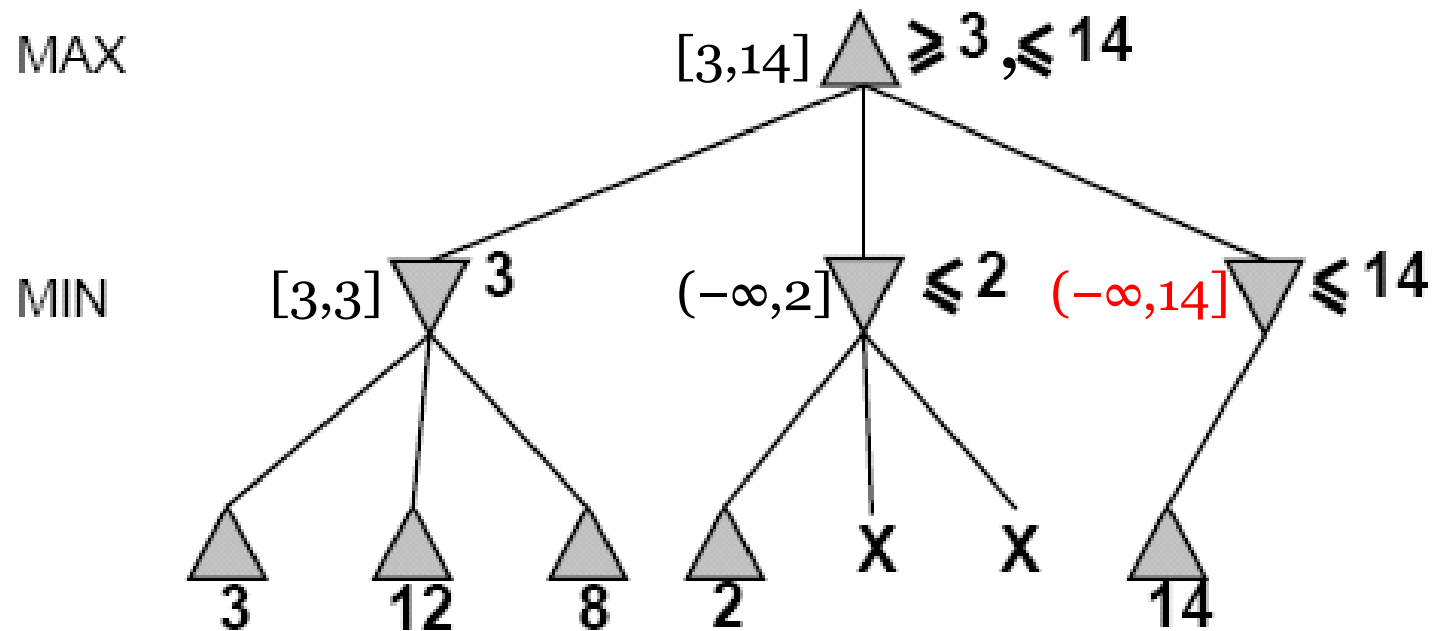
Alpha-Beta Pruning

- Example



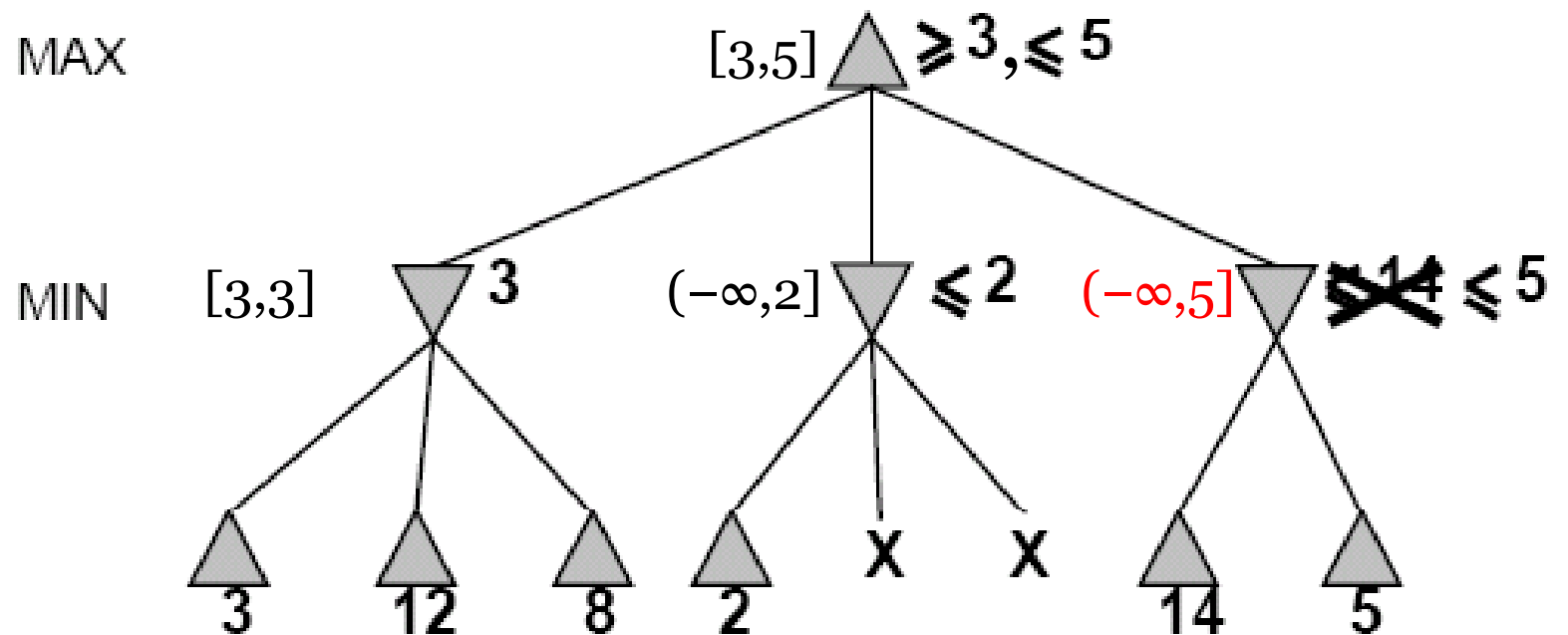
Alpha-Beta Pruning

- Example



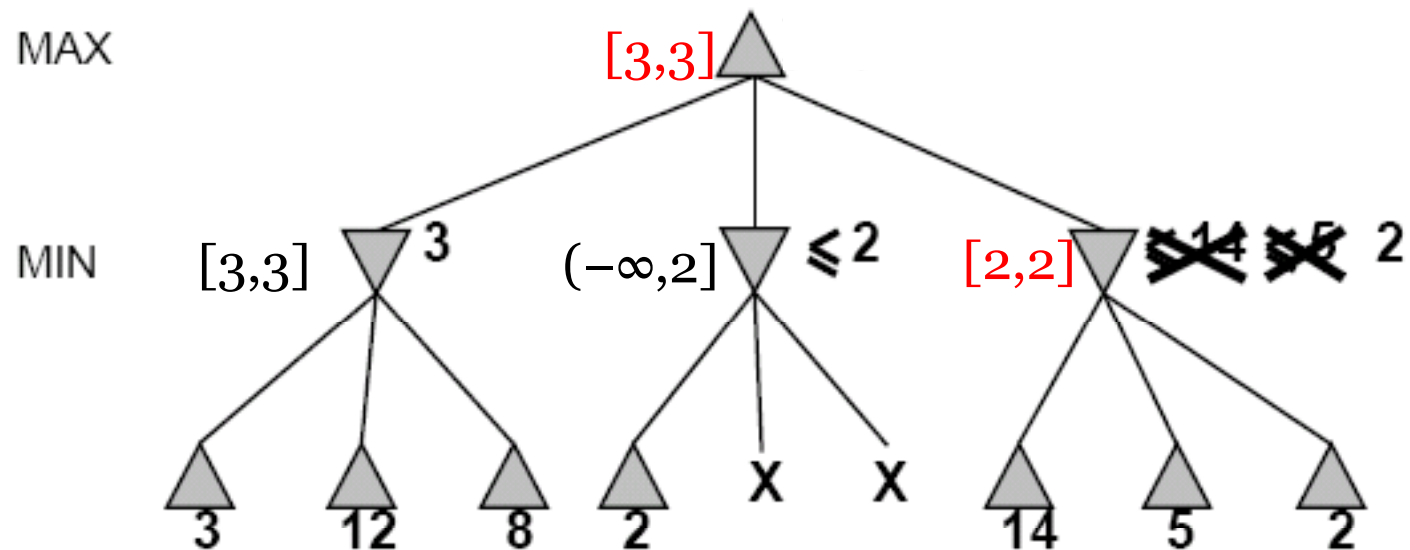
Alpha-Beta Pruning

- Example



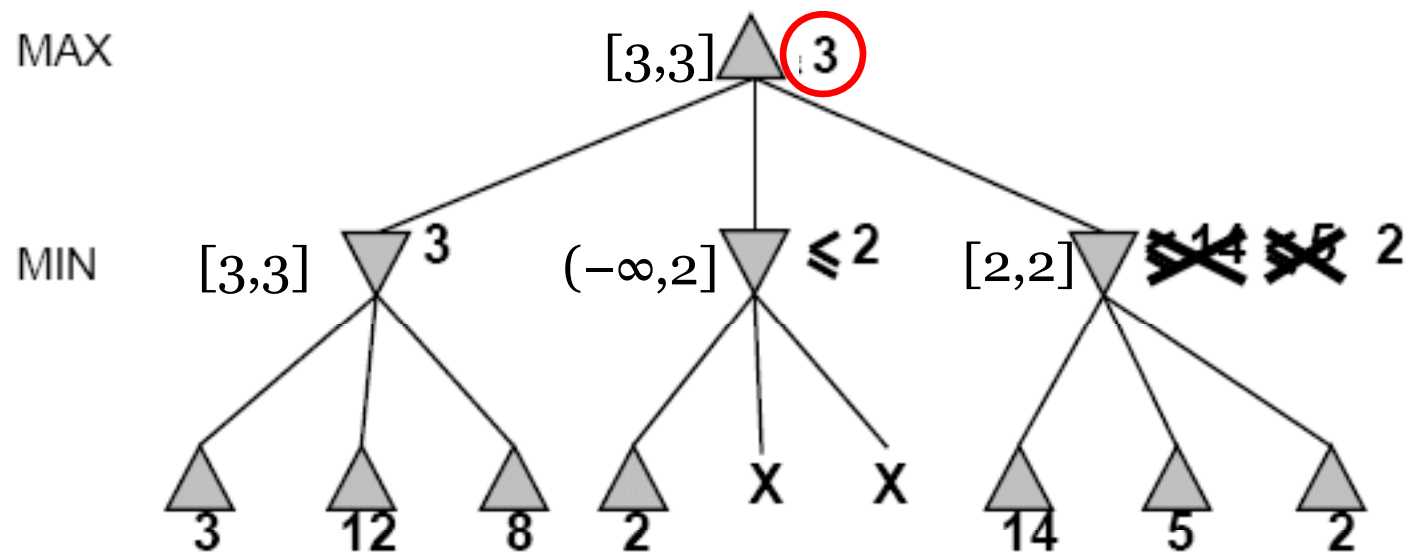
Alpha-Beta Pruning

- Example



Alpha-Beta Pruning

- Example



Alpha-Beta Pruning

- **Effectiveness of Alpha-beta pruning**

- ◇ Alpha-Beta is guaranteed to compute the same value for the root node as computed by **Minimax**.
- ◇ **Worst case:** NO pruning, examining $O(b^d)$ leaf nodes, where each node has b children and a **d-ply** search is performed
- ◇ **Best case:** examine only $O(b^{(d/2)})$ leaf nodes.
- ◇ You can search **twice as deep as Minimax!** Or the branch factor is $b^{(1/2)}$ rather than b .

Alpha-Beta Pruning

- **Effectiveness of Alpha-beta pruning**
 - ◇ **Best case** is when each player's best move is the leftmost alternative, i.e. at MAX nodes the child with the largest value generated first, and at MIN nodes the child with the smallest value generated first.
 - ◇ In **Deep Blue**, they found empirically that with Alpha-Beta pruning the average branching factor at each node was about **6** instead of about **35-40**

Alpha-Beta Pruning

- **Effectiveness of Alpha-beta pruning**

- ◇ It is guaranteed to return exactly the same value as the Min-Max algorithm. It is a pure optimization without any approximations or tradeoffs.
- ◇ In a perfectly ordered tree, with the best moves on the left, alpha beta reduces the cost of the search from order **bd** to order **$b^{(d/2)}$** , that is, we can search twice as deep.
- ◇ Worst case it won't prune any nodes.

Outline

- Game Theory
- Types of Games
- Game Trees
- Minimax Search
- Alpha-Beta Pruning
- **Summary**

Summary

- Game playing is best modeled as a search problem
- Game trees represent alternate computer/opponent moves
- Evaluation functions estimate the quality of a given board configuration for the Max player.
- Minimax is a procedure which chooses moves by assuming that the opponent will always choose the move which is best for them
- Alpha-Beta is a procedure which can prune large parts of the search tree and allow search to go deeper

References

1. A. Colman, “Experimental Games”, In Cooperation and Competition in Human and Animals. A. Colman (Ed.), Can Nostrand Reinhold, 1982.
2. S. Hart and A. Mas-Colell, “Introduction”, In Cooperation: Game-Theoretic Approaches. S. Hart and A. Mas-Colell (Eds.), NATO ASI Series, Series F: Computer and Systems Sciences, vol. 55, 1994.
3. M. Kamel, ECE457A: Cooperative and Adaptive Algorithms. Spring 2011, University of Waterloo.
4. Yun Peng. Ch.6 Game Playing: Adversarial Search from CMSC471-Artificial Intelligence, University of Maryland Baltimore County.
5. M. T. Jones, *Artificial Intelligence: A Systems Approach*.
6. O. Basir, ECE457A: Cooperative and Adaptive Algorithms. Spring 2013, University of Waterloo.